

AG35-CET&AG35-EUT QuecOpen(SDK) 低功耗模式开发指导

LTE Standard 模块系列

版本：1.0.1

日期：2024-12-04

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

文档历史

本文档为临时版本，其中涉及的模块引脚或接口定义、频段、功能、特性及设计等（若有）发生变动的可能性较大，部分参数经过初步验证或仍未验证。变更或补充信息将在后续正式版本中体现。临时版本仅供临时参考使用，若与正式版本存在差异，应以最新版本为准。修订记录如下所示：

修订记录

版本	日期	作者	变更表述
-	2023-11-14	Gabriel LI	文档创建
1.0.0	2023-11-14	Gabriel LI	临时版本
			临时版本
1.0.1	2024-12-04	Gabriel LI	1. 新增适用模块 AG35-EUT。 2. 新增普通 GPIO 中断阻止休眠的说明(第 4.2.6 章)。 3. 更新使用 AT 命令设置搜网间隔时间的最大值（第 4.3.2 章）。

目录

文档历史	3
目录	4
表格索引	5
图片索引	6
1 引言	7
2 低功耗模式	8
2.1. 低功耗模式介绍	8
2.2. GPIO 中断唤醒方案	9
2.2.1. SLEEP_SYS_IND	9
2.2.2. WAKEUP_OUT	9
2.2.3. WAKEUP_IN	10
2.2.4. 低功耗模式 GPIO 配置	10
3 低功耗模式 API	12
3.1. 头文件	12
3.2. 函数概览	12
3.3. 函数详解	13
3.3.1. 低功耗模式设置 API	13
3.3.1.1. ql_lpm_init	13
3.3.1.2. ql_lpm_deinit	14
3.3.1.3. ql_autosleep_enable	15
3.3.2. 唤醒锁 API	15
3.3.2.1. ql_slp_wakelock_create	15
3.3.2.2. ql_slp_wakelock_lock	16
3.3.2.3. ql_slp_wakelock_unlock	16
3.3.2.4. ql_slp_wakelock_destroy	17
4 常见问题处理	18
4.1. 无法进入低功耗模式	18
4.2. 唤醒原因	19
4.2.1. VBus 中断唤醒	20
4.2.2. RTC 中断唤醒	20
4.2.3. WAKEUP_IN 中断唤醒	21
4.2.4. PWRKEY 唤醒	21
4.2.5. 电话/短信/网络消息等中断唤醒	22
4.2.6. 普通 GPIO 中断阻止休眠	23
4.3. 低功耗模式下的异常唤醒	24
4.3.1. 大量异常唤醒导致耗流高	24
4.3.2. 无网或弱网环境频繁唤醒	25
5 附录 参考文档及术语缩写	26

表格索引

表 1: SLEEP_SYS_IND 电平状态指示	9
表 2: WAKEUP_OUT 电平状态指示（建议方案）	9
表 3: WAKEUP_IN 信号边沿状态指示	10
表 4: 设备树中 GPIO 配置说明	10
表 5: 函数概览	12
表 6: 参考文档	26
表 7: 术语缩写	26

Preliminary
Quectel Confidential

图片索引

图 1: 低功耗模式进入/退出流程	7
图 2: 低功耗模式状态图	8
图 3: GPIO 中断唤醒方案.....	9
图 4: 查看 autosleep 使能状态	18
图 5: USB_VBUS 唤醒锁	18
图 6: lpm 程序唤醒锁.....	18
图 7: 系统未持有唤醒锁	19
图 8: VBUS 中断唤醒 Log	20
图 9: RTC0 中断唤醒 Log.....	20
图 10: RTC1 中断唤醒 Log.....	20
图 11: WAKEUP_IN 中断唤醒 Log.....	21
图 12: PWRKEY 唤醒 Log	21
图 13: CP 侧中断唤醒 Log	22
图 14: AP 侧中断唤醒 Log	22
图 15: 查看配置了中断的引脚	23
图 16: GPIO 中断唤醒 Log.....	24
图 17: 默认搜网机制.....	25

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen®方案；QuecOpen®是基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen®方案，主要介绍基于 Linux 的 autosleep 和 wakelock 技术开发的 QuecOpen®低功耗模式及唤醒方案。

- **autosleep**

autosleep 从 Android wakelocks 补丁集演化而来，基于唤醒源（wakeup source）实现；通过 autosleep，模块可达到 Opportunistic suspend 的效果，即系统不持有任何 wakelock 时，可自动进入低功耗模式。

- **wakelock**

即唤醒锁机制。autosleep 开启后，只要系统所有的唤醒锁均被释放，则模块可进入低功耗模式；若任一唤醒锁未被释放，则无法进入低功耗模式。

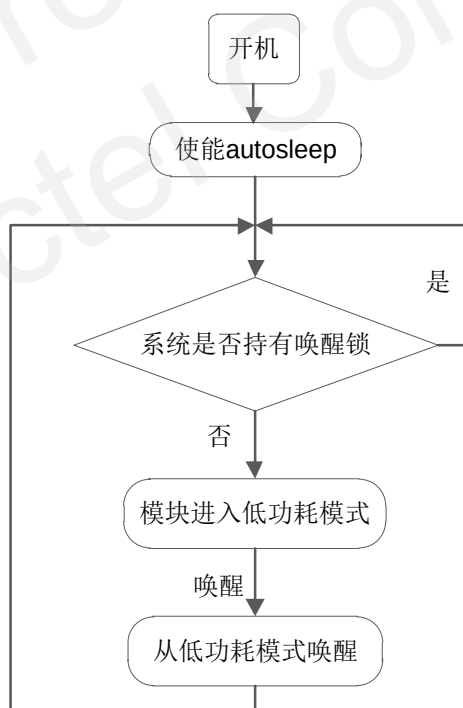


图 1：低功耗模式进入/退出流程

2 低功耗模式

2.1. 低功耗模式介绍

低功耗模式（即 LPM，Low Power Mode）的状态图如下所示。当 MCU/Android 检测车辆 CAN 信号发现熄火（即 ACC OFF）时，将通过 SPI、UART 或 USB 等接口通知模块停止用户 App 的各项业务、停止外设工作、释放相应的唤醒锁。当模块内部所有唤醒锁均被释放且外设均未工作时，模块将进入休眠，即低功耗模式。

当语音、数据、短信、定时器或者特定 GPIO 中断到来时，模块会从低功耗模式唤醒。相应的回调函数或者处理函数锁定相应的唤醒锁后，模块进入正常工作状态。

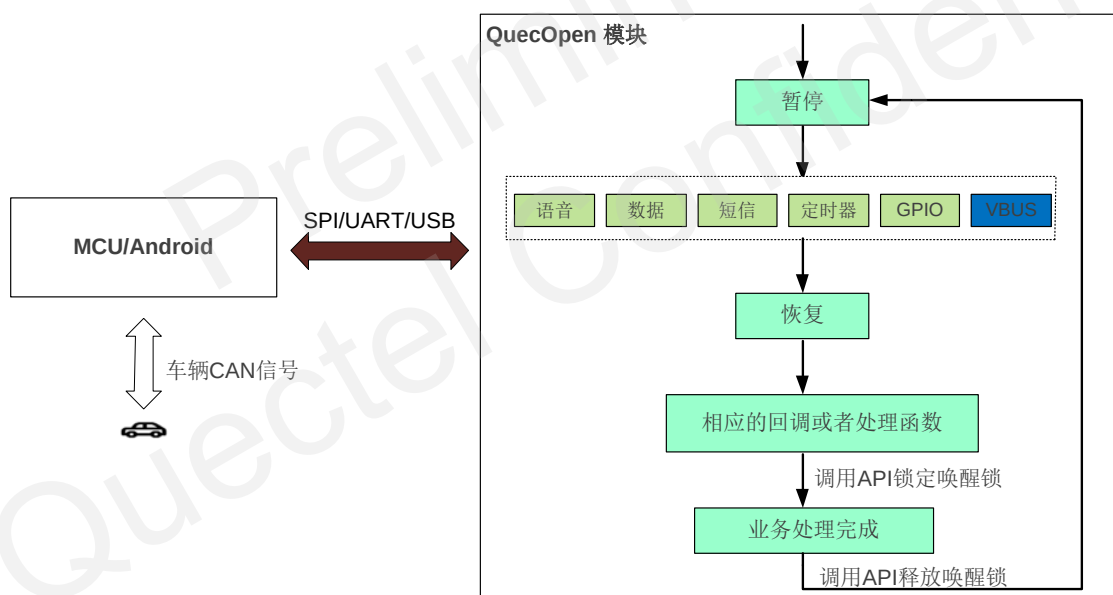


图 2：低功耗模式状态图

备注

建议模块控制的外设在模块从低功耗模式唤醒时正常供电。若外设必须要在低功耗模式下去电、唤醒时上电，则要保证外设供电的控制权在模块侧，以避免外部 MCU 等控制器上电和模块内驱动进入低功耗模式时间不同步，造成驱动休眠失败。

2.2. GPIO 中断唤醒方案

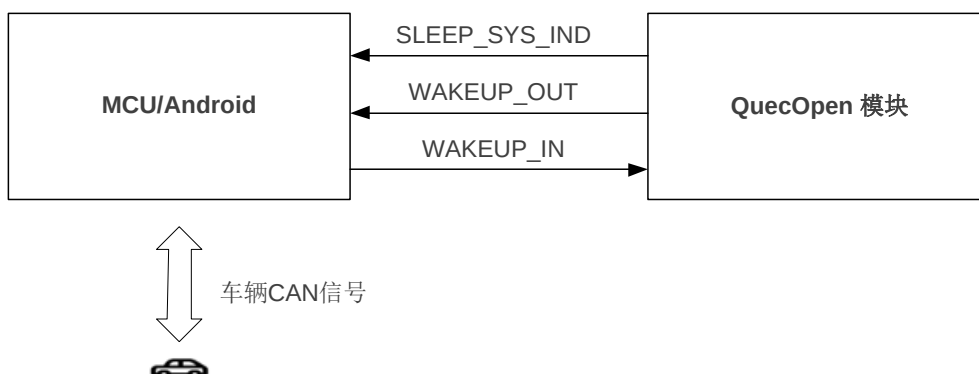


图 3: GPIO 中断唤醒方案

2.2.1. SLEEP_SYS_IND

该引脚由模块驱动注册管理，指示模块进入或退出低功耗模式。

表 1: SLEEP_SYS_IND 电平状态指示

电平	状态指示
0	默认表示模块进入低功耗模式
1	默认表示模块退出低功耗模式

2.2.2. WAKEUP_OUT

该引脚由用户层注册管理，指示用户应用程序进入或退出低功耗模式。建议设计方案如下：

表 2: WAKEUP_OUT 电平状态指示（建议方案）

电平	状态指示
0	表示用户应用程序进入低功耗模式
1	表示用户应用程序退出低功耗模式

2.2.3. WAKEUP_IN

该引脚由模块驱动注册管理，指示由外部 MCU 控制的模块低功耗模式/唤醒状态。

表 3: WAKEUP_IN 信号边沿状态指示

边沿	状态指示
下降沿	默认表示 MCU 控制模块进入低功耗模式
上升沿	默认表示 MCU 唤醒模块

2.2.4. 低功耗模式 GPIO 配置

设备树配置文件 `ql-ol-kernel/arch/arm/boot/dts/quectel/ql-1806e-common.dtsi` 中 GPIO 配置示例如下：

```
quec,gpio_lpm{
    compatible = "quec,ql_lpm";
    pinctrl-names = "default","sleep";
    pinctrl-0 = <&wakeup_in_pin &sleep_sys_ind>;
    pinctrl-1 = <&wakeup_in_pin_sleep>;
    gpios = <&gpio 118 0>,          // wakeup_in
           <&gpio 38 0>;           //sleep_sys_ind
    ql,gpio-names = "wakeup_in","sleep_sys_ind";
    ql,sleep-sys-ind-enable;
    ql,sleep-sys-ind-state = <0>;
    status = "ok";
};
```

表 4: 设备树中 GPIO 配置说明

参数	参数描述	默认配置
<i>gpios</i>	WAKEUP_IN 和 SLEEP_SYS_IND（即 <i>ql,gpio-names</i> ）的引脚配置	118 WAKEUP_IN 引脚配置（对应模块的引脚 61，即 GPIO2）
		38 SLEEP_SYS_IND 引脚配置（对应模块的引脚 147，即 GPIO5）
<i>ql,sleep-sys-ind-enable</i>	低功耗模式状态指示引脚（SLEEP_SYS_IND）是否使能	开启
<i>ql,sleep-sys-ind-state</i>	0 模块进入低功耗模式时指示低电平，唤醒时指示高电平	0

	1	模块进入低功耗模式时指示高电平，唤醒时指示低电平	
status	"ok"	开启低功耗模式	"ok"
	"disabled"	关闭低功耗模式	

Preliminary
Quectel Confidential

3 低功耗模式 API

3.1. 头文件

低功耗模式 API 的头文件为 `ql_lpm.h` 和 `ql_sleep_wakelock.h`，位于 `ql-sysroots/usr/include/ql-sdk` 目录下。若无特别说明，本文档所涉及头文件均在该目录下。

低功耗模式 API 的使用可以参考 SDK 示例程序文件 `example_lpm.c`，该示例程序文件位于 `sample/low_power_consume_app/` 目录下。

3.2. 函数概览

表 5：函数概览

函数	说明
<code>ql_lpm_init()</code>	初始化低功耗模式
<code>ql_lpm_deinit()</code>	注销低功耗模式
<code>ql_autosleep_enable()</code>	使能 autosleep
<code>ql_slp_wakelock_create()</code>	创建唤醒锁
<code>ql_slp_wakelock_lock()</code>	锁定已创建的唤醒锁
<code>ql_slp_wakelock_unlock()</code>	释放指定的唤醒锁
<code>ql_slp_wakelock_destroy()</code>	注销唤醒锁

3.3. 函数详解

3.3.1. 低功耗模式设置 API

3.3.1.1. ql_lpm_init

该函数用于初始化低功耗模式。

- 函数原型

```
int ql_lpm_init(ql_lpm_handler_t ql_lpm_handler)
```

- 参数

ql_lpm_handler:

[In] 用户回调。当 WAKEUP_IN 信号边沿状态发生变化时将触发该回调函数；详见第 3.3.1.1.1 章。

- 返回值

0 函数执行成功

-1 函数执行失败

备注

重复调用该函数时，仅执行最后一次调用所传入的回调函数，因此不建议重复调用该函数。

3.3.1.1.1. ql_lpm_handler_t

当 WAKEUP_IN 的信号边沿状态发生变化时将触发该回调函数；该回调函数由 *ql_lpm_init()* 注册。

- 函数原型

```
typedef void (*ql_lpm_handler_t)(ql_lpm_edge_t lpm_edge)
```

- 参数

lpm_edge:

[In] 底层上报的 WAKEUP_IN 信号边沿状态变化；详见第 3.3.1.1.2 章。

- 返回值

无

3.3.1.1.2. ql_lpm_edge_t

WAKEUP_IN 信号边沿状态变化枚举定义如下：

```
typedef enum
{
    E_QL_LPM_FALLING = 0,
    E_QL_LPM_RISING = 1,
}ql_lpm_edge_t
```

- 成员

成员	描述
<i>E_QL_LPM_FALLING</i>	下降沿（WAKEUP_IN 电平下降时，模块进入低功耗模式）
<i>E_QL_LPM_RISING</i>	上升沿（WAKEUP_IN 电平上升时，从低功耗模式唤醒模块）

3.3.1.2. ql_lpm_deinit

该函数用于注销低功耗模式。

- 函数原型

```
int ql_lpm_deinit(void)
```

- 参数

无

- 返回值

0 函数执行成功
-1 函数执行失败

3.3.1.3. ql_autosleep_enable

该函数用于使能 `autosleep`。使能后，模块会在满足条件（所有唤醒锁均被释放且外设均未工作）时自动进入低功耗模式。

- 函数原型

```
int ql_autosleep_enable(char enable)
```

- 参数

enable:

[In] 是否使能 `autosleep`。

0 关闭 `autosleep`

1 使能 `autosleep`

- 返回值

0 函数执行成功

-1 函数执行失败

备注

调用 `ql_autosleep_enable(1)`使能 `autosleep` 后，一般不建议使用 `ql_autosleep_enable(0)`保持系统唤醒，而应该调用 `ql_slp_wakelock_lock()`使系统保持唤醒；调用 `ql_slp_wakelock_unlock()`可以释放唤醒锁，使模块能够再次进入低功耗模式；详情请参考第 3.3.2.2 章和第 3.3.2.3 章。模块进入低功耗模式后系统进程会被冻结，唤醒后进程会继续运行。

3.3.2. 唤醒锁 API

3.3.2.1. ql_slp_wakelock_create

该函数用于创建唤醒锁。此函数创建的唤醒锁所有者为当前进程；每次调用函数只能创建一个唤醒锁，最多可以创建 512 个唤醒锁。创建多个唤醒锁时，唤醒锁名称不能重复。

- 函数原型

```
int ql_slp_wakelock_create(const char *name, size_t len)
```

- 参数

name:

[In] 唤醒锁名称（在 `/sys/kernel/debug/wakeup_sources` 文件内体现）。

len:

[In] 唤醒锁名称的长度。最大长度为 28 个字符。

- 返回值

大于 0 唤醒锁描述符，函数执行成功

-1 函数执行失败

3.3.2.2. **ql_slp_wakelock_lock**

该函数用于锁定已创建的唤醒锁。锁定后模块无法进入低功耗模式。

- 函数原型

```
int ql_slp_wakelock_lock(int fd)
```

- 参数

fd:

[In] 唤醒锁描述符。

- 返回值

0 函数执行成功

-1 函数执行失败

3.3.2.3. **ql_slp_wakelock_unlock**

该函数用于释放指定的唤醒锁。如果系统内其他唤醒锁均已被释放，且 App 已使能 **autosleep** 功能，那么模块将会进入低功耗模式。

- 函数原型

```
int ql_slp_wakelock_unlock(int fd)
```

- 参数

fd:

[In] 唤醒锁描述符。

- 返回值
- 0 函数执行成功
- 1 函数执行失败

3.3.2.4. ql_slp_wakelock_destroy

该函数用于注销唤醒锁。

- 函数原型

```
int ql_slp_wakelock_destroy(int fd)
```

- 参数

fd:

[In] 唤醒锁描述符。

- 返回值
- 0 函数执行成功
- 1 函数执行失败

4 常见问题处理

4.1. 无法进入低功耗模式

运行 `cat /sys/power/autosleep` 命令查看模块是否已使能 `autosleep`。

情况一： 若模块未使能 `autosleep`，可调用 `ql_autosleep_enable()`函数使能。下图示例查看结果为 `mem`，表示 `autosleep` 已使能。

```
~ #
~ # cat /sys/power/autosleep
mem
~ #
```

图 4：查看 `autosleep` 使能状态

情况二： 若 `autosleep` 已使能但未进入低功耗模式，请使用如下命令查看当前系统持有的唤醒锁：

```
awk '$7!= 0 {print $1" "$7}' /sys/kernel/debug/wakeup_sources
```

如下图所示，若输出 `c0000000.usb`，则表示 `USB_VBUS` 引脚为高电平，需拉低 `USB_VBUS` 引脚才能使模块进入低功耗模式。

```
root@OpenWrt:~#
root@OpenWrt:~# awk '$7!= 0 {print $1" "$7}' /sys/kernel/debug/wakeup_sources
name active since
c0000000.usb 3198886
root@OpenWrt:~#
```

图 5：USB_VBUS 唤醒锁

如下图所示，若输出 `ql_lpm`，则表示 `lpm` 程序持有唤醒锁，需释放唤醒锁使模块进入低功耗模式。

```
root@OpenWrt:/data# awk '$7 !=0 {print $1" "$7}' /sys/kernel/debug/wakeup_sources
ces
name active_since
3063.ql_lpm 4719
root@OpenWrt:/data#
```

图 6：lpm 程序唤醒锁

如下图所示，若输出如下信息，则表示系统未持有唤醒锁，模块可以进入低功耗模式。

```
root@openwrt:/data# awk '$7 !=0 {print $1" "$7}' /sys/kernel/debug/wakeup_sources
name active_since
root@openwrt:/data#
```

图 7：系统未持有唤醒锁

备注

若输出信息为其他唤醒锁标识符，用户可自行排查对应的应用程序内是否存在唤醒锁。只有将应用程序内所有的唤醒锁都释放了，模块才能正常进入低功耗模式。

4.2. 唤醒原因

可以使用模块的调试串口抓取 log 信息，进而对模块唤醒原因进行分析，确认是否为非预期唤醒并针对性地处理。

模块进入低功耗模式后，首先执行如下命令打开调试串口 log，即可查看唤醒中断信息。

```
echo 9 > /proc/sys/kernel/printk
echo 16371 > /dev/msocket_dump
echo N > /sys/module/printk/parameters/console_suspend
```

然后，根据第 4.2.1~4.2.6 章流程排查唤醒原因。如需移远通信技术支持分析唤醒原因，请提供低功耗模式唤醒时的 Kernel Log；若为网络消息唤醒，请提供 CP Log。

备注

有关抓取 CP Log 的详情，请联系移远通信技术支持。

4.2.1. USB_VBUS 中断唤醒

USB_VBUS 作为唤醒源唤醒的日志如下图所示：

```
[ 133.214673] APCR:0x32204000, CPCR:0xfec86000
[ 133.214673] APSR:0xa5000000, CPSR:0xa5000000
[ 133.214673] General xPCR :0xfec86000
[ 133.214673] Power Mode Status :0xdf
[ 133.214673] =====
[ 133.214673] =====quectel_wakeup:=====
[ 133.214734] =====vbus=====
[ 133.216106] dwc2 c0000000.usb: dwc2 resume
[ 133.218546] Change ddr freq to saved value. (cur: 266000Khz)
[ 133.284060] dwc2 c0000000.usb: asr-usb vbus interrupt is served..
```

图 8: VBUS 中断唤醒 Log

4.2.2. RTC 中断唤醒

模块支持 RTC0 和 RTC1 两个设备。RTC0 作为唤醒源唤醒的日志如下图所示：

```
[ 82.625385] =====
[ 82.625385] =====quectel_wakeup:=====
[ 82.625415] PM: pm_system_irq_wakeup: 20 triggered ASRPM802
[ 82.626788] dwc2 c0000000.usb: dwc2 resume
[ 82.627947] =====rtc0=====
[ 82.628923] =====rtc0=====
[ 82.631119] Change ddr freq to saved value. (cur: 266000Khz)
[ 82.924101] emac_phy_connect: phy@0
[ 82.924131] ==> set eamc interface: rgmii
```

图 9: RTC0 中断唤醒 Log

RTC1 作为唤醒源唤醒的日志如下图所示：

```
[ 50.114649] General xPCR :0xfec86000
[ 50.114649] Power Mode Status :0xdf
[ 50.114649] =====
[ 50.114649] =====quectel_wakeup:=====
[ 50.114679] PM: pm_system_irq_wakeup: 22 triggered rtc Alarm
[ 50.116052] dwc2 c0000000.usb: dwc2 resume
[ 50.116418] =====rtc1=====
[ 50.116479] =====rtc1=====
[ 50.118644] Change ddr freq to saved value. (cur: 266000Khz)
[ 50.413548] emac_phy_connect: phy@0
[ 50.413609] ==> set eamc interface: rgmii
```

图 10: RTC1 中断唤醒 Log

4.2.3. WAKEUP_IN 中断唤醒

WAKEUP_IN 是模块的唤醒引脚，默认为 GPIO_117，唤醒日志如下图所示：

```
[ 927.494388] APSR:0xa5000000, CPSR:0xa5000000
[ 927.494388] General xPCR :0xfee86000
[ 927.494388] Power Mode Status :0xdf
[ 927.494388] =====
[ 927.494388] =====quectel_wakeup:=====
[ 927.495760] dwc2 c0000000.usb: dwc2 resume
[ 927.496400] =====wakeup_in=====
[ 927.497895] =====wakeup_in=====
[ 927.498383] Change ddr freq to saved value. (cur: 266000Khz)
[ 927.498505] =====wakeup_in=====
[ 927.584576] =====wakeup_in=====
[ 927.584942] =====wakeup_in=====
[ 927.593512] =====wakeup_in=====
[ 927.593665] =====wakeup_in=====
```

图 11: WAKEUP_IN 中断唤醒 Log

4.2.4. PWRKEY 唤醒

按下 PWRKEY 键唤醒模块时唤醒日志如下所示：

```
[ 122.533183] Power Mode Status :0xdf
[ 122.533183] =====
[ 122.533183] =====quectel_wakeup:=====
[ 122.533214] PM: pm_system_irq_wakeup: 20 triggered ASRPM802
[ 122.534800] dwc2 c0000000.usb: dwc2 resume
[ 122.535959] =====power_key=====
[ 122.536355] pm80x_exton1n press = 1
[ 122.538582] Change ddr freq to saved value. (cur: 266000Khz)
[ 122.638835] =====power_key=====
[ 122.639201] pm80x_exton1n press = 0
```

图 12: PWRKEY 唤醒 Log

4.2.5. 电话/短信/网络消息等中断唤醒

常见电话、短信、网络消息等中断唤醒的日志中会打印相应的寄存器值。

CP 侧中断唤醒的日志打印如下。需结合第 4.2.6 章所述日志进一步判断。

```
APCR:0x32204000, CPCR:0xf8c82000
APSR:0x34000000, CPSR:0x34000000
```

示例如下：

```
[ 201.294705] AWUCRM:0x3020090, CWUCRM:0x7107810
[ 201.294705] General wakeup source mask:0x7127890
[ 201.294705] APCR:0x32204000, CPCR:0xf8882000
[ 201.294705] APSR:0x34000000, CPSR:0x34000000
[ 201.294705] General xPCR :0xtaa86000
[ 201.294705] Power Mode Status :0xdf
[ 201.294705] =====
[ 201.294705] =====quectel_wakeup:=====
[ 201.296169] dwc2 c0000000.usb: dwc2 resume
[ 201.298639] Change ddr freq to saved value. (cur: 266000Khz)
```

图 13: CP 侧中断唤醒 Log

AP 侧中断唤醒的日志打印：

```
APCR:0x32204000, CPCR:0xfec86000
APSR:0xa5000000, CPSR:0xa5000000
```

示例如下：

```
[ 359.214551] AWUCRM:0x3020090, CWUCRM:0x7107812
[ 359.214551] General wakeup source mask:0x7127892
[ 359.214551] APCR:0x32204000, CPCR:0xfe886000
[ 359.214551] APSR:0xa5000000, CPSR:0xa5000000
[ 359.214551] General xPCR :0xtea86000
[ 359.214551] Power Mode Status :0xdf
[ 359.214551] =====
[ 359.214551] =====quectel_wakeup:=====
[ 359.214581] PM: pm_system_irq_wakeup: 20 triggered ASRPM802
[ 359.215954] dwc2 c0000000.usb: dwc2 resume
[ 359.217082] =====power_key=====
```

图 14: AP 侧中断唤醒 Log

备注

具体的网络端唤醒场景需通过 CP Log 排查。有关抓取 CP Log 的详情，请联系移远通信技术支持。

4.2.6. 普通 GPIO 中断阻止休眠

模块进入低功耗模式时，需将不用于唤醒模块的 GPIO 的 IRQ 中断关闭，即在驱动的挂起（suspend）回调中关闭 GPIO 中断，在唤醒（resume）回调中重新使能 GPIO 中断。在普通 GPIO 中断阻止休眠的场景下，打印的日志会指向 CP 侧唤醒，需进一步判断是 CP 侧中断唤醒还是 GPIO 中断阻止休眠。

执行如下命令查看已被申请作为中断输入引脚的 GPIO。如下图所示，带 GPIO 标签的引脚即为配置了中断的引脚。

```
cat /proc/interrupts
```

```
root@OpenWrt:/# cat /proc/interrupts
CPU0
18:         6   icu_irq    2 Edge      asr-spi.1
19:         0   icu_irq    3 Edge      asr-spi.0
20:         0   icu_irq    4 Edge      ASRPM802
21:         0   icu_irq    5 Edge      rtc 1Hz
22:         0   icu_irq    6 Edge      rtc Alrm
23:         0   icu_irq    7 Edge      pxa2xx-i2c.0
26:         0   icu_irq   10 Edge      eth0
27:         0   icu_irq   11 Edge      eth0
29:       121165   icu_irq   13 Edge      local-timer
35:         0   icu_irq   19 Edge      UART4
36:         613   icu_irq   20 Edge      pxa9xx-ACIPIC0
42:         4   icu_irq   26 Edge      devfreq-ddr
43:         0   icu_irq   27 Edge      UART1
44:         0   icu_irq   28 Edge      UART3
46:         0   icu_irq   30 Edge      seh
55:        577   icu_irq   39 Edge      mmc1, mmc0
60:        332   icu_irq   44 Edge      c0000000.usb, c0000000.usb
61:     258553   icu_irq   45 Edge      d420b000.spi
63:     18484   icu_irq   47 Edge      pdma
65:         24   icu_irq   49 Edge      gpio-mux
69:         2   icu_irq   53 Edge      asr-usb-vbus
70:        440   icu_irq   54 Edge      pxa2xx-i2c.2
72:         0   icu_irq   56 Edge      d403d100.ripc1, ripc_wakeup
75:         0   icu_irq   59 Edge      UART2
83:         0   icu_irq   67 Edge      pxa2xx-i2c.3
214:        47   GPIO    118 Edge      wakeup_in_irq
215:         0   GPIO    119 Edge      us im0
216:         0   GPIO    120 Edge      pps.-1
224:         0   ASRPM802    0 Edge      rtc
225:         0   ASRPM802    1 Edge      exton1n
Err:         0
root@OpenWrt:/#
```

图 15: 查看配置了中断的引脚

GPIO_120 阻止休眠的日志如下所示，显示为 CP 侧阻止休眠。

```
APCR:0x32204000, CPCRC:0xf8882000
APSR:0x34000000, CPSR:0x34000000
```

再结合如下 Log 分析，当 GEDR 后打印的全部为 0x0 时，可判断为 CP 侧唤醒。当出现如下图红框中的打印，说明存在 GPIO 触发了中断唤醒或阻止了系统休眠，GEDR[96-127]: 0x1000000 表示 GPIO 号在 96 至 127 之间的引脚（即 GPIO_120）触发 1 路中断。用户可根据配置排查是否需要在进入休眠时关闭中断。

```
kernel: [ 168.422759] =====wake up events status =====
kernel: [ 168.422759] BEFORE SUSPEND AWUCRS:0x90080000
kernel: [ 168.422759] Power Mode Status :0x0
kernel: [ 168.422759] After SUSPEND
kernel: [ 168.422759] AWUCRS:0xf6180002, CWUCRS:0xf6180002
kernel: [ 168.422759] General wakeup source status:0xf6180002
kernel: [ 168.422759] AWUCRM:0x3020090, CWUCRM:0x7107810
kernel: [ 168.422759] General wakeup source mask:0x7127890
kernel: [ 168.422759] APCR:0x32204000, CPCR:0xf8882000
kernel: [ 168.422759] APSR:0x34000000, CPSR:0x34000000
kernel: [ 168.422759] General xPCR :0xfaa86000
kernel: [ 168.422759] =====
kernel: [ 168.422759] =====quectel_wakeup:=====
kernel: [ 168.422759] I/TC: After SUSPEND^M
kernel: [ 168.422759] I/TC: AWUCRS:0xf6180002, CWUCRS:0xf6180002^M
kernel: [ 168.422759] I/TC: General wakeup source status:0xf6180002^M
kernel: [ 168.422759] I/TC: AWUCRM:0x3020094, CWUCRM:0x17107812^M
kernel: [ 168.422759] I/TC: General wakeup source mask:0x17127896^M
kernel: [ 168.422759] I/TC: APCR:0xbe086000, CPCR:0xfe886000^M
kernel: [ 168.422759] I/TC: General xPCR :0xfe886000^M
kernel: [ 168.422759] I/TC: Power Mode Status :0xdf^M
kernel: [ 168.422759] I/TC: AP Sub system is woken up by:3G Base band^M
kernel: [ 168.422759] I/TC: AP is woken up by AP_INT_ASYNC^M
kernel: [ 168.422759] I/TC: INTReg_1: 0x20000 ^M
kernel: [ 168.422759] I/TC: Possible Wakeup by IRQ: GPIO_AP^M
kernel: [ 168.422759] GPLR[00-031]: 0x80100300
kernel: [ 168.422759] GEDR[00-031]: 0x0
kernel: [ 168.422759] GPLR[32-063]: 0x600081
kernel: [ 168.422759] GEDR[32-063]: 0x0
kernel: [ 168.422759] GPLR[64-095]: 0x0
kernel: [ 168.422759] GEDR[64-095]: 0x0
kernel: [ 168.422759] GPLR[96-127]: 0x1C00008
kernel: [ 168.422759] GEDR[96-127]: 0x1000000
kernel: [ 168.447383] TS not enabled
kernel: [ 168.507609] TS not enabled
kernel: [ 168.567603] TS not enabled
kernel: [ 168.627596] TS not enabled
kernel: [ 168.676335] dwc2 vbus off, lx_state: 3
kernel: [ 168.676426] dwc2 c0000000.usb: dwc2 resume
kernel: [ 168.677524] spidev_read_work spidev_sync_read_ext fail, ret = -108
kernel: [ 168.679690] Change ddr freq to saved value. (cur: 533000Khz)
```

图 16: GPIO 中断唤醒 Log

4.3. 低功耗模式下的异常唤醒

4.3.1. 大量异常唤醒导致耗流高

模块在低功耗模式下被异常唤醒时，若 AP 侧无硬件故障等问题，则唤醒源均可正常打印，此时需抓取 CP Log 进行唤醒源信息分析。若执行 **AT+CFUN=0** 后，异常唤醒消失，此时可调用 `ql_nw_set_power_mode()` 设置低功耗模式，屏蔽非预期的信息上报。调用函数的详细信息及支持的低功耗模式请参考文档[2]。

4.3.2. 无网或弱网环境频繁唤醒

QOOS（Quectel Out Of Service）是移远通信为弱信号和无信号环境下的休眠场景提供的可定制化的低功耗优化方案。此方案尤其适用于车载或其他设备休眠时地理位置不变的应用场景。

以车载产品为例，汽车可能会在地下车库、偏远地区等弱信号、无信号环境下停靠较长时间，此时很可能因信号弱而无法注网。默认的搜网机制会较为频繁地搜网（搜网最大间隔 20 秒，且每次搜网时间不可控），造成休眠功耗超出车载标准。QOOS 机制在默认搜网机制基础上，对掉网后的搜网机制进行优化，实现搜网间隔和每次搜网时间可控，客户可根据具体需求定制休眠状态下的搜网方案，使设备休眠耗流达到预期标准。因此，建议客户在设备进入休眠前修改 QOOS 机制下的搜网间隔，在退出休眠后恢复 QOOS 机制默认配置。默认的搜网机制如下图所示：

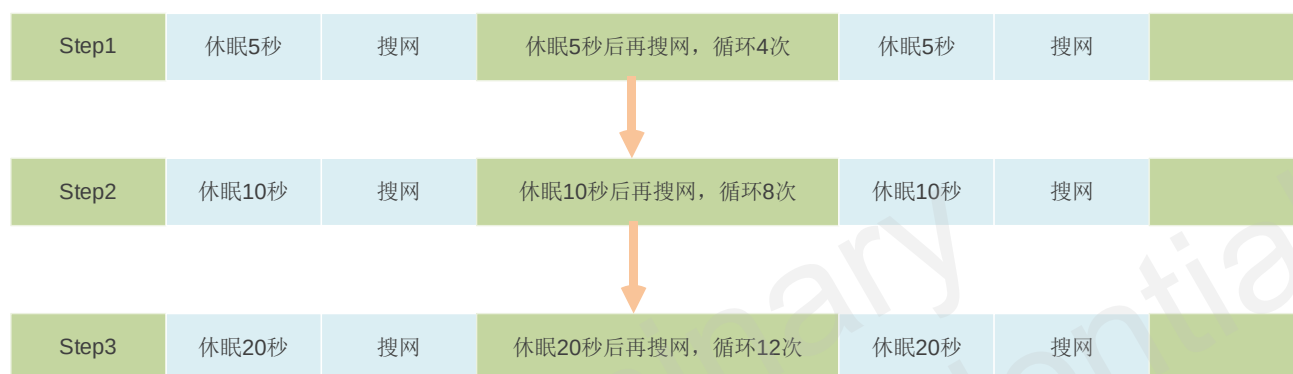


图 17：默认搜网机制

模块支持通过使用 AT 命令设置搜网间隔时间：

- **AT+QCFG="qoos",0**
- **AT+QCFG="qoos",1,<interval1>,<interval2>,<interval3>**

使用默认搜网间隔
分别设置三个步骤的搜网间隔。
单位：秒。最大值：600 秒。

5 附录 参考文档及术语缩写

表 6: 参考文档

文档名称
[1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导
[2] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_蜂窝网络信息_开发指导

表 7: 术语缩写

缩写	英文全称	中文全称
ACC	Adaptive Cruise Control	自适应巡航控制
API	Application Program Interface	应用程序接口
App	Application	应用
CAN	Controller Area Network	控制器局域网
CP	Control Program	控制程序
GPIO	General-purpose input/output	通用型之输入输出
IoV	Internet of Vehicles	车联网
LPM	Low Power Mode	低功耗模式
MCU	Microcontroller Unit	微控制单元
SDK	Software Development Kit	软件开发工具包
SPI	Serial Peripheral Interface	串行外设接口
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发传输器
USB	Universal Serial Bus	通用串行总线